

NAN 2026 「YAKI SEASON」 AI 활용 기술 문서

1. 핵심 요약

YAKI SEASON의 AI 활용 핵심은 결과물을 많이 생성하는 것이 아니라, AI가 만든 결과를 사람이 검증하고 책임질 수 있는 게임 제작물로 전환하는 체계를 구축한 것이다.

프로젝트는 다음 두 개의 제작 파이프라인으로 구성된다.

“ 기획·개발 파이프라인 ”

사람의 아이디어를 요구사항, 역할별 태스크, 코드·데이터·테스트와 플레이 가능한 기능으로 전환한다.

“ 아트 생성·승격 파이프라인 ”

AI 생성 이미지를 후처리, 개별 검수, 실제 화면 검수와 사람 승인을 거쳐 추적 가능한 런타임 자산으로 전환한다.

두 파이프라인에는 역할별 작업 범위, SHA-256, provenance, 자동 테스트와 사람 승인을 공통 통제층으로 적용했다. AI가 결과물을 제안하고 제작 속도를 높이지만, 게임의 방향과 품질은 사람이 결정한다. 근거, 자동 검증과 사람 승인이 없는 AI 결과는 게임에 반영하지 않는다.

2. 프로젝트와 AI 활용 요약

‘YAKI SEASON’은 할아버지의 야키토리 노포를 물려받아 꼬치를 직접 조립하고 숯불에 구우며 손님을 운영하는 웹 기반 타임 매니지먼트 게임이다. 두 명의 팀원이 23일 동안 기획, 개발, 데이터, 이야기, 아트와 QA를 병렬로 진행했다.

AI 활용 목표는 다음과 같다.

- 사람의 아이디어를 잃지 않고 구현 가능한 요구사항으로 전환한다.
- 두 명의 팀이 여러 전문 역할을 병렬로 수행할 수 있게 한다.
- 생성 이미지의 원본, 도구, 후처리와 승인 상태를 추적한다.
- 빠르게 제작한 코드와 아트가 실제 게임에서 깨지지 않도록 검증한다.
- AI의 추측, 과도한 수정과 허위 완료 보고를 구조적으로 제한한다.

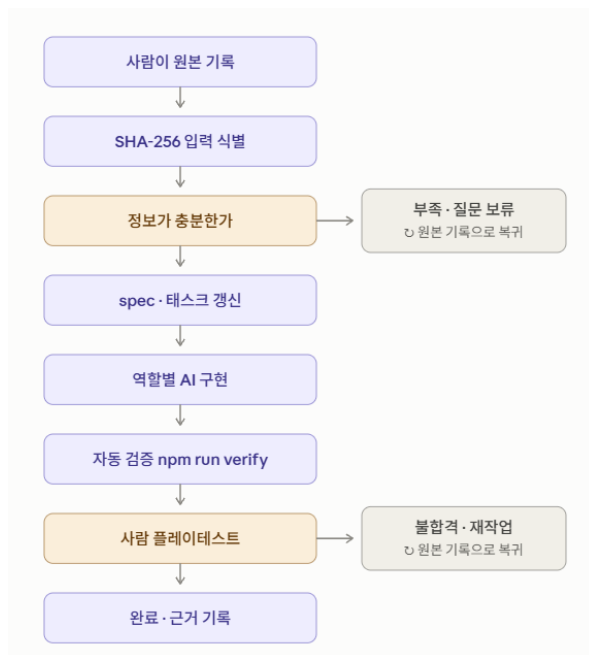
3. 사용 도구와 역할

도구	주요 활용
Claude Code	요구사항·태스크 구조화, 게임 코드·데이터·테스트 작성
OpenAI Codex	코드·문서 작성 보조, 이미지 생성 작업 구성과 검증 실행
Codex 내장 imagegen / image_gen	배경·캐릭터·음식·스테이션 이미지 생성 및 편집
ChatGPT 이미지 생성	편차가 큰 캐릭터와 개별 자산 후보 생성

Three.js / GLTFExporter	웹 게임 렌더링과 승인 이미지 기반 3D 자산화
Pillow와 고정 후처리 도구	배경 제거, 크롭, 크기·파일 규격 정규화
Vitest / Playwright	게임 규칙, 통합 흐름과 실제 브라우저 회귀 검증

생성형 AI는 아이디어, 코드와 이미지처럼 탐색이 필요한 작업에 사용했다. 배경 제거, 해시 계산, 스키마 검사와 테스트는 같은 입력에서 같은 결과가 나오는 결정적 자동화로 처리했다.

4. 파이프라인 A : 기획과 개발



진행 과정

사람 원본 기록

- SHA-256으로 입력 식별
- AI 요구사항 구조화와 모순 탐지
- 역할별 태스크와 쓰기 범위 배정
- 코드·데이터·테스트 동시 작성
- 자동 검증
- 사람 플레이테스트
- 완료 근거 기록 또는 변경 요청 재입력

핵심 운영 원칙

- 사람이 `docs/inbox/`에 기록한 원본은 AI가 임의로 고치거나 삭제하지 않는다.

- 같은 경로와 SHA의 원본은 중복 처리하지 않고, SHA가 바뀌면 새로운 입력으로 판단한다.
- 정보가 부족하거나 충돌하면 AI가 임의로 확정하지 않고 질문 또는 보류 상태로 남긴다.
- 완료된 과거 태스크를 고쳐 쓰지 않고 후속 태스크를 만든다.
- 한 파일은 한 시점에 한 역할만 수정한다.
- 기능 코드만 작성하지 않고 테스트와 완료 근거를 함께 남긴다.

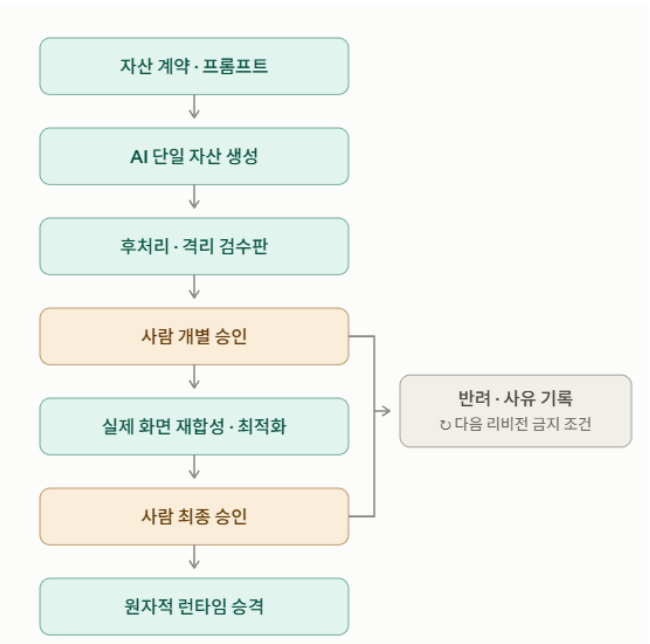
AI 역할 분리

역할	책임
PM	상태, 의존관계, 우선순위와 사람 승인 큐 관리
개발자 1	게임 규칙, 하루 영업, 조립·굽기·서빙과 저장
개발자 2	공개 화면, 이야기와 승인 아트 연결
개발자 3	콘텐츠 밸런스 데이터, JSON schema와 회귀 도구
아티스트 1	조립·그릴·음식 모델과 셰이더
아티스트 2	가게 외관·소품·인물 초상
아티스트 3	드링크·서빙·정리·정산 아트
통합 QA	재현, 회귀 검증과 공개 차단 gate

대표 개발 사례

첫 손님의 네기마 두 개를 동시에 묶인 애니메이션으로 처리하지 않고, 각 꼬치가 독립적인 배치 시점, 앞면·뒷면 타이머, 뒤집기와 회수 상태를 갖도록 구현했다. 단위 테스트와 FHD·720p E2E로 두 꼬치가 서로 간섭하지 않는지 검증하고, 마지막에는 사람이 실제 화면에서 조작감을 확인했다.

5. 파이프라인 B : 아트 생성과 승격



진행 과정

자산 계약과 프로젝트 고유 아트 기준

- AI 단일 자산 생성
- 배경 제거·크롭·규격 정규화
- 체커보드 격리 검수
- 사람 개별 승인
- 실제 게임 화면 FHD·720p 재합성
- 최적화
- 사람 최종 승인
- 원자적 런타임 승격과 회귀 검증

핵심 운영 원칙

- 특정 작가를 모방하지 않고 `YS-HANDCRAFTED-NIGHT-v1` 프로젝트 고유 기준을 사용한다.
- 한 프롬프트에서는 한 개의 독립 자산만 생성한다.
- 생성 원본은 게임이 직접 읽지 않으며 별도 작업 공간에 보관한다.
- 반려 이유를 다음 리비전의 금지 조건으로 상속한다.
- AI가 스스로 `approved-by-user`를 기록할 수 없다.
- 개별 자산 승인과 실제 소비 화면 최종 승인을 별도 gate로 운영한다.

Provenance와 안전한 등록

각 자산에는 다음 정보를 기록한다

- 에셋 ID, 리비전과 사용 화면
- 생성 도구와 프롬프트 요약
- 참조 원본과 결과 파일의 SHA-256
- 후처리 도구와 파라미터
- 해상도, 용량과 파일 형식
- 금지 요소와 반려 이력
- 사람 승인 날짜와 근거

검수를 통과한 자산은 finalizer가 만든 `runtime-handoff.json`과 일회성 영수증을 사용해 파일과 manifest를 하나의 트랜잭션으로 등록한다. 하나라도 실패하면 전체 변경을 복구한다.

6. 검증과 사람 승인

자동 검증

npm run verify

└─ 런타임 에셋 파일·manifest·schema 검사

- └─ 기준 이미지 36장 SHA-256-viewport 검사
- └─ Vitest 단위·통합 테스트
- └─ Playwright Chromium FHD·720p E2E

테스트 결과에는 실행 커밋 SHA와 dirty 상태를 함께 남긴다. 코드가 바뀌면 과거 테스트 결과를 현재 빌드의 통과 증거로 사용하지 않는다. 간헐적 실패는 최초 실패와 재시도 결과를 모두 기록한다.

사람만 할 수 있는 판단

- 게임의 방향과 우선순위 결정
- 충돌하는 요구사항의 최종 선택
- 아트와 시나리오의 개별·최종 승인
- 실제 플레이를 통한 재미와 조작감 판정
- 외부 제출, 공개와 원격 반영 결정

자동 테스트가 모두 통과해도 사람이 직접 플레이하기 전에는 완료로 판단하지 않는다.

7. 실제 게임에 반영된 결과

플레이 기능

- 시작 화면, 새 게임·이어하기와 프롤로그
- 네기마 재료 순서 조립
- 복수 꼬치의 독립적인 앞·뒷면 굽기, 뒤집기와 회수
- Three.js-GLSL 기반 연속 익힘 색 표현
- 생맥주 제조와 음식·음료 부분 제공
- 복수 손님·주문·대기·반응과 좌석 정리
- 하루 영업 마감, 정산과 다음 영업일 전환
- S0 프롤로그, D1~D3 캠페인과 D4 예고
- 캠페인 저장·복구와 콘텐츠·밸런스 데이터

제작 체계

- 원본 SHA에서 요구사항·태스크까지 이어지는 추적 기록
- 역할과 수정 경로가 분리된 8개 AI 역할
- 생성 아트의 provenance와 두 단계 사람 승인
- 실제 게임 화면 FHD·720p 재합성 검수
- 원자적 런타임 에셋 승격과 실패 복구
- 단위·통합·E2E와 시각 레퍼런스 검증

8. 실제 게임에 반영된 결과

실패에서 개선한 사례

전체 화면 생성 방식 폐기

초기에는 전체 콘셉트 화면을 한 장으로 생성한 뒤 배경, 인물과 도구를 잘라 쓰러 했다. 그러나 가려진 경계와 원근이 맞지 않아 재작업이 반복됐다. 실패를 숨기지 않고 사용 중지 근거로 남긴 뒤, 한 프롬프트에서 한 자산만 생성하고 실제 게임 화면에서 다시 합성하는 방식으로 전환했다.

네기마 익힘 표현 개선

단계별 이미지를 교체하는 방식은 실제 플레이에서 색 변화가 부자연스러웠다. 구현 편의보다 조리 감각을 우선해, 승인된 네기마 원본 한 장에 GLSL 셰이더로 익힘 색을 연속 적용하는 방식으로 변경했다.

9. 결론

`YAKI SEASON`은 AI에게 게임 전체를 한 번에 맡긴 프로젝트가 아니다. 기획·개발과 아트 생성·승격이라는 두 제작 파이프라인을 만들고, 그 위에 역할 분리, 파일 경계, SHA-256, provenance, 자동 테스트와 사람 승인을 배치했다.

이 구조를 통해 두 명의 팀원이 23일 동안 게임 규칙, 캠페인, 데이터, 아트와 QA를 병렬로 진행하면서도 다음 질문에 답할 수 있게 했다.

- 이 요구사항은 어떤 사람 원본에서 나왔는가?
- 이 코드와 그림은 어떤 AI와 프롬프트로 만들었는가?
- 누가 언제 승인했는가?
- 실제 게임 화면에서 검증했는가?
- 문제가 생기면 어느 단계로 돌아가야 하는가?